Open in app ↗

Medium Search Write

🚀 LangChain Deep Dive: Building Modular LLM Applications with Python

Tejasrisomarouthu 5 min read · 1 day ago

🧠 Introduction: Why Simple LLM Apps Fail

Have you ever built a chatbot that forgets everything after one message?
Or tried connecting an AI model to real-world tools like search or calculators

— and got stuck?

That's because raw LLMs are powerful.... but incomplete.

They don't :

- Remember past conversations
- Use external tools
- Handle multi-step reasoning

👉 And that's exactly where LangChain comes in.

🔍 What is LangChain ?

LangChain is a framework that helps developers build **modular, intelligent applications powered by Large Language Models(LLMs).**

Instead of making a single API call, LangChain allows you to:

- Chain multiple steps together
- Add memory to conversations
- Integrate external tools (search, APIs, databases)
- Build dynamic, decision-making agents

👉 In simple terms:
LangChain turns LLMs into complete AI systems.

⚡ Why LangChain is important ?

Modern AI applications are not just about aswering questions.

They require:

- Context awareness
- Real-time data access
- Decision-making ability
- Structured workflows

LangChain solves these problems through:

- **Orchestration** → managing workflows
- **Chaining** → connecting multiple steps
- **Tool Integration** → using APIs & services
- **Memory** → maintaining context

🧠 How LangChain Works Internally

- LangChain follows a modular pipeline architecture where each component performs a specific role.
- Data flows sequentially through prompts, models, chains, and tools.

- This design enables flexibility, as developers can replace or modify individual components without affecting the entire system

👉 This gives you **extra edge over others**

🧩 Core Components of LangChain

Let's break LangChain into its building blocks 👇

🤖 1. LLMs and Chat Models

💡 Concept

LLMs are the brain of your application.

🧠 Why it exists

They process input and generate human-like responses

💻 Code Example

```
from langchain_openai import ChatOpenAI

#Initialize model
llm = ChatOpenAI(model="gpt-4o-mini")

#Invoke model
response = llm.invoke("Explain LangChain in simple terms")
print(response.content)
```

👉 Key Insights :

LLMs generate output, but they don’t manage workflows or memory on their own

📝 2. Prompt Templates

💡 Concept

Prompts define how you communicate with the model

🧠 Why it exists

To make prompts reusable and dynamic

💻 Code Example

```
from langchain.prompts import PromptTemplate

template = PromptTemplate(
input_variables = ["topic"]
template = "Explain {topic} like I'm a beginner"
)

prompt = template.format(topic="LangChain")
print(prompt)
```

👉 Key Insights :

Prompt Templates = reusable instructions for your AI

🔗 3. Chains

💡 Concept

Chains connect multiple steps into a workflow

🧠 Why it exists

To automate multi-step processing

💻 Code Example

```
from langchain.chains import LLMChain

chain = LLMChain(llm=llm, prompt=template)

result = chain.run("Artificial Intelligence")
print(result)
```

👉 Think of Chains as assembly line for AI tasks.

🧠 4. Memory

💡 Concept

Stores previous interactions.

🧠 Why it exists

To create context-aware conversations.

💻 Code Example

```
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain

memory = ConversationBufferMemory()

conversation = ConversationChain(
llm=llm,
```

```
memory=memory
)

print(conversation.run("Hi, my name is Tejasri"))
print(conversation.run("what is my name?"))
```

👉 Key Insight:

Without memory, your AI behaves like it has amnesia

🤖 5. Agents

💡 Concept

Agents decode what action to take

🧠 Why it exists

To enable reasoning + tool usage

💻 Code Example

```
from langchain.agents import initialize_agent, load_tools

tools = load_tools(["serpapi","llm-math"],llm=llm)

agent = initialize_agent(
tools,
llm,
agent="zero-shot-react-description"
)

agent.run("What is 25*4?")
```

👉 Powerful Insight:

Agents don't just respond — they think and act.

🛠️ 6. Tools

💡 Concept

External functionalities used by agents.

Examples:

- Google search
- Calculator
- APIs

👉 Tools extend AI beyond knowledge → into action.

📄 7. Document Loaders

💡 Concept

Load external data into your application

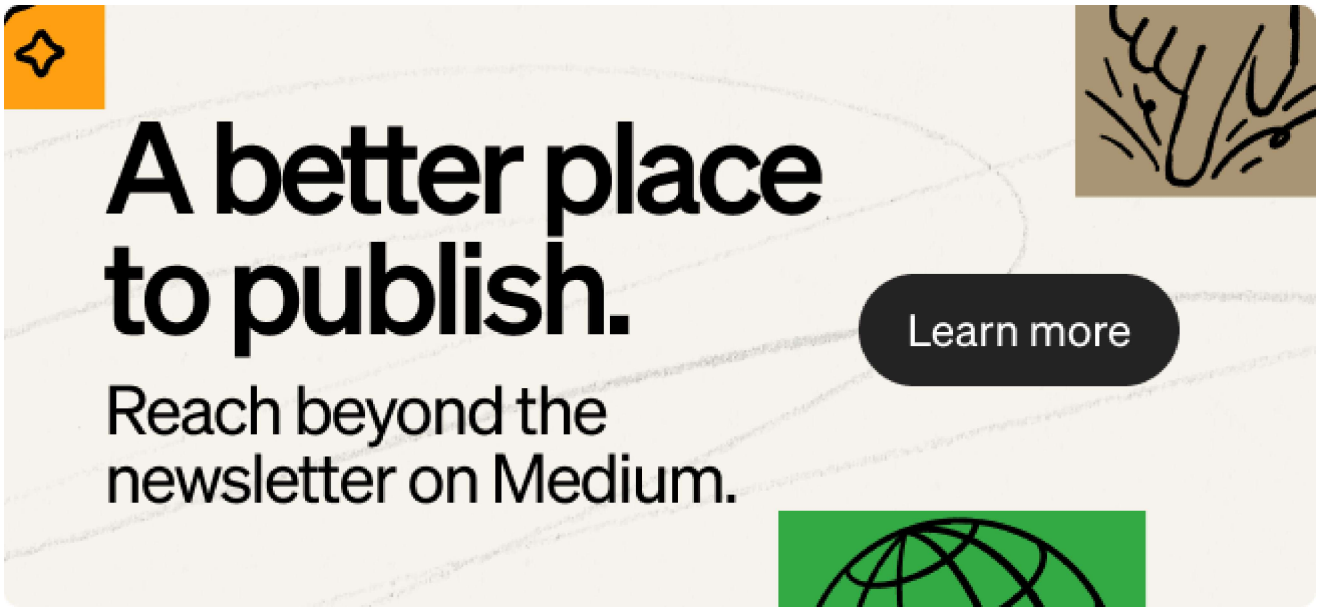
💻 Code Example

```
from langchain.document_loaders import TextLoader

loader = TextLoader("data.txt")
documents = loader.load()
```

👉 Used for building document-based AI systems.

🔍 8. Vector Stores (Indexes)



💡 Concept

Store embeddings for semantic search.

🧠 Why it exists

To retrieve relevant data efficiently

💻 Code Example

```
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings

embeddings = OpenAIEmbeddings()

vectorstore = FAISS.from_documents(documents, embeddings)
```

👉 This powers **RAG (Retrieval-Augmented Generation)** systems.

👾 Architecture of LangChain

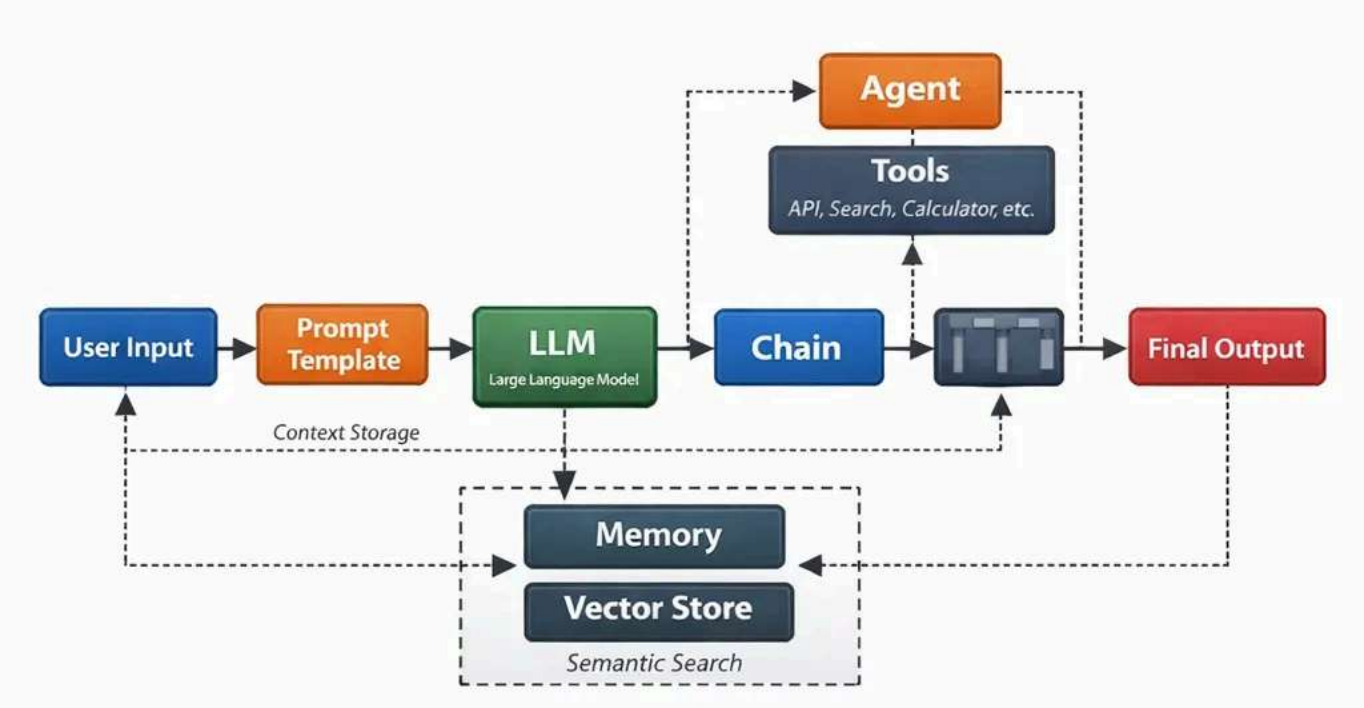
Let's understand how everything connects 👇

🔄 **Flow :**

User Input → prompt → LLM → Chain → Agent/Tool → Output

👉 When a user provides input, it is first structured using a prompt template.
The LLM processes this prompt and generates an intermediate response.
Chains manage multi-step workflows, while agents decide whether external tools are required.
Finally, tools fetch or compute additional information before producing the final output.

📊 Diagram (Flow chart)



💻 Hands-on Implementation

✅ Basic LLM Call

```
response = llm.invoke("What is Artificial Intelligence?")
print(response.content)
```

✅ Prompt + Chain

```
chain.run("Machine Learning")
```

✅ Memory Example

```
conversation.run("I love coding")
conversation.run("What do I love?")
```

✅ Agent example

agent.run("What is square root of 144?")

🌍 Real-World Use Cases

🔹 1. AI Chatbot with Memory

Traditional chatbots fail because they don't retain previous context. LangChain memory stores conversation history and feeds it back to the model, enabling more natural and human-like responses.

Problem: Chatbots forget context
Solution: Use Memory

Components Used:

- LLM
- Memory
- Chain

👉 **Result:** Context-aware conversations

🔹 2. Document Q&A System

Extracting useful information from large documents like PDFs or research papers is challenging and time-consuming.

LangChain solves this using embeddings and vector stores, enabling semantic search and retrieval of relevant information.

Problem: Extract info from large documents
Solution: Use embeddings + vector search

Components Used:

- Document Loader
- Vector Store
- Retriever

👉 **Used in:** PDFs, research papers, legal docs

🔹 3. **AI Research Assistant**

Users often require real-time and accurate information, which static LLM responses cannot always provide.
LangChain enables agents to use external tools like search APIs and calculators to fetch up-to-date data and perform computations.

Problem: Need real-time answers
Solution: Use agents with tools

Components Used:

- Agent
- Tools

- LLM

👉 **Result:** AI that can search + calculate + answer

⚖️ Advantages & Limitations

✅ Advantages

- Modular architecture
- Easy integration
- Rapid prototyping
- Supports multiple LLM providers

❌ Limitations

- High latency
- Debugging complexity
- API costs

🚫 When NOT to use LangChain

Avoid LangChain when:

- You need simple one-step responses

- Low latency is critical
- You are building small scripts

🧠 Key Takeaways

- LangChain transforms LLMs into full AI systems
- It enables chaining, memory, and tool usage
- It simplifies building complex AI workflows

🚀 Future Scope

The future of LangChain includes:

- **LangGraph** for advanced workflows
- Multi-agent systems
- Autonomous AI applications

🎯 Conclusion

We are moving from :

👉 Asking AI questions
➡️ To

👉 Building AI Systems

And LangChain is the bridge that makes it possible

💬 Final Thought

If you understand LangChain, you're not just using AI anymore.....

👉 You're engineering intelligence

💻 Code Repository

🔗 **GitHub Repository:**
https://github.com/Tejasri016/Gen-AI-Assignments-

This repository contains well-structured Python implementations demonstrating key LangChain components such as LLMs, Prompt Templates, Chains, Memory, and Agents.

📌 *Note: This repository is part of my GenAI Internship and is continuously updated with additional examples and improvements as I explore more advanced use cases.*

Langchain | Generative Ai Tools | Python | Artificial Intelligence | Machine Learning

Written by Tejasrisomarouthu

1 follower · 8 following

Edit profile

No responses yet

Tejasrisomarouthu

What are your thoughts?

More from Tejasrisomarouthu

Tejasrisomarouthu

Tejasrisomarouthu

🚀 NLP Token Classification Explained: NER, POS Tagging,...

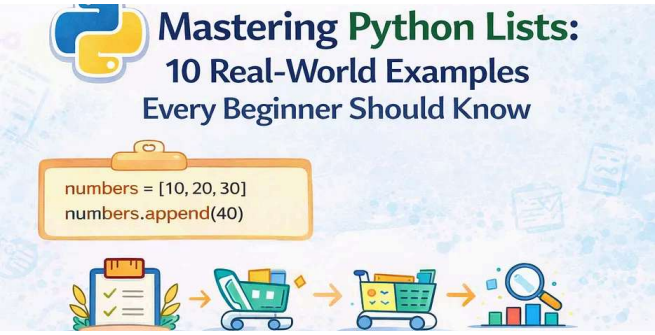
Learn how machines understand text word by word using token classification techniques...

Apr 5 1

From Raw Text to Machine Understanding: A Complete NLP...

Introduction to NLP

Mar 29 5



Tejasrisomarouthu

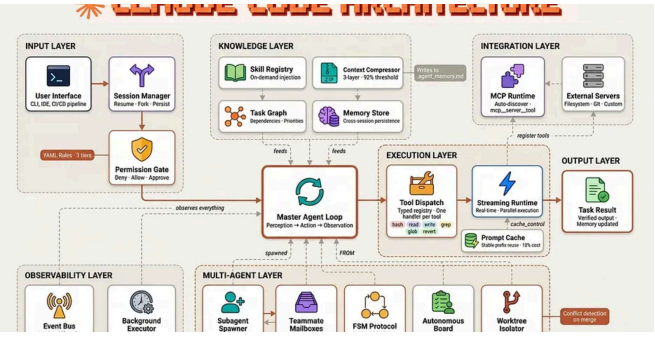
Mastering Python Lists : 10 Real-World Examples Every Beginner...

When you start learning Python, lists are one of the most powerful and beginner friendly...

Feb 16 31

See all from Tejasrisomarouthu

Recommended from Medium

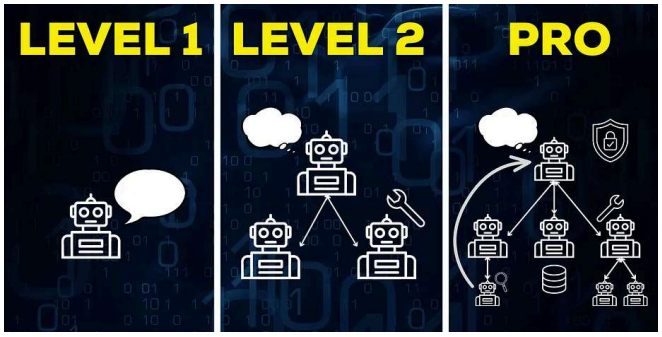


In Level Up Coding by Fareed Khan

Building Claude Code with Harness Engineering

Multi-agents, MCP, skills system, context pipelines and more

Apr 6 1K 8



In Data Science Collective by Marina Wyss

AI Agents: Complete Course

From beginner to intermediate to production.

Dec 6, 2025 5.7K 231

Leo Godin

Claude Code is Great

You Just Need to Learn How to Use It

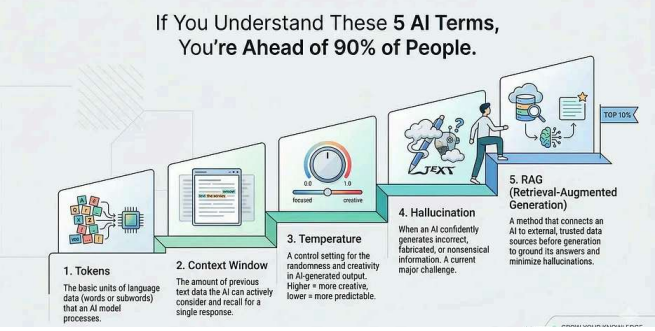
Mar 2 2.1K 53

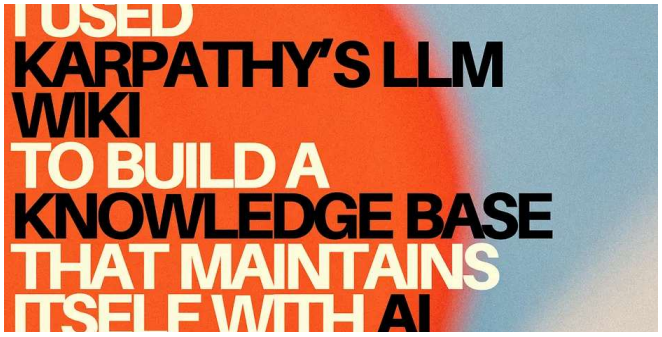
Joe Njenga

Claude Code Ultraplan Launched: I Just Tested It (And It's Better Tha...

Anthropic has added Claude Code ultraplan, and I was quick to test it. You might like it or...

Apr 5 744 23





In Towards AI by Shreyas Naphad

If You Understand These 5 AI Terms, You're Ahead of 90% of...

Master the core ideas behind AI without getting lost

Mar 29 9.4K 190

Balu Kosuri

I used Karpathy's LLM Wiki to build a knowledge base that maintains...

Here is a scenario most knowledge workers know too well.

6d ago 231 2

See more recommendations